Week 2, Day 6 (Weekend): Spring Security, JWT, REST, Exceptions and Validation

Prepared for Upendra Kumar Sharma | 5–6 hours | Main topic: Spring Security filter chain, JWT, OAuth2, method security, REST best practices, exception handling, validation | Plus: build a small secured API

Today's Schedule

- **0:00–1:15** Study the Spring Security architecture and configuration (sections 1 and 2).
- **1:15–2:15** Study JWT and OAuth2/OIDC (sections 3 and 4).
- **2:15–2:45** Study method-level security and general web security (section 5).
- **2:45–3:45** Study REST best practices, exception handling and validation (sections 6 to 8).
- **3:45–4:45** Hands-on: build the mini secured API in section 9.
- **4:45–5:15** Answer the interview questions aloud (section 10) and add 10 flashcards.
- **5:15–5:45** Light DSA revision: redo one problem each from binary search, stacks, intervals and linked lists, with a 10-minute limit each.

This guide targets **Spring Boot 3 with Spring Security 6**. Older tutorials use WebSecurityConfigurerAdapter and antMatchers, which no longer exist.

1. Spring Security Architecture

Authentication vs authorization

- **Authentication:** who are you? **Authorization:** what are you allowed to do?
- A failed authentication returns **401 Unauthorized**. An authenticated user lacking permission gets **403 Forbidden**.

The filter chain

- Spring Security is a chain of servlet filters. A DelegatingFilterProxy hands each request to **FilterChainProxy**, which picks the first matching **SecurityFilterChain** and runs its filters in order.
- **Important filters, roughly in order:** SecurityContextHolderFilter (loads the security context), HeaderWriterFilter (security headers), CsrfFilter, LogoutFilter, UsernamePasswordAuthenticationFilter (form login), BearerTokenAuthenticationFilter

(JWT with resource server support), BasicAuthenticationFilter, AnonymousAuthenticationFilter, **ExceptionTranslationFilter**, and **AuthorizationFilter** (the final access decision).

- **ExceptionTranslationFilter** turns security exceptions into responses: an AuthenticationEntryPoint produces the 401, and an AccessDeniedHandler produces the 403.
- A custom JWT filter is usually a OncePerRequestFilter added **before UsernamePasswordAuthenticationFilter**.

Core objects

- **SecurityContextHolder** stores the **SecurityContext** for the current thread (ThreadLocal), which holds the **Authentication**.
- **Authentication:** principal (who), credentials (proof, cleared after login), authorities (what they may do), and an authenticated flag.
- **AuthenticationManager** (implemented by ProviderManager) delegates to one or more **AuthenticationProviders**. The usual one, **DaoAuthenticationProvider**, uses a **UserDetailsService** to load the user and a **PasswordEncoder** to check the password.
- **GrantedAuthority:** a permission or role string. A role is an authority with the **ROLE_ prefix**, so hasRole('ADMIN') checks for ROLE_ADMIN, while hasAuthority('ROLE_ADMIN') uses the full string.
- **Passwords:** store only salted hashes. Use **BCryptPasswordEncoder** or the DelegatingPasswordEncoder (the Boot default), or Argon2. Never store or log plain passwords.

2. Configuring Security (Spring Security 6)

- Define a **SecurityFilterChain bean** with the lambda DSL.
- **authorizeHttpRequests** replaces authorizeRequests, and **requestMatchers** replaces antMatchers and mvcMatchers.
- **Order matters:** the first matching rule wins, so put specific rules before general ones, and end with anyRequest().authenticated().
- **Stateless APIs:** set the session creation policy to STATELESS. **CSRF protection** can be disabled for pure token-based APIs that do not use cookies for authentication, but must stay enabled for browser apps that rely on session cookies.
- **CORS:** configure allowed origins explicitly through a CorsConfigurationSource. Never use a wildcard origin together with credentials.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http, JwtAuthFilter jwtFilter)
throws Exception {
        http
            .csrf(csrf -> csrf.disable())                       // stateless
token API
            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**", "/actuator/health").permitAll()
                .requestMatchers(HttpMethod.DELETE,
"/api/**").hasRole("ADMIN")
                .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration
cfg) throws Exception {
        return cfg.getAuthenticationManager();
    }
}
```

3. JWT Authentication

What a JWT is

- Three Base64URL parts separated by dots: **header.payload.signature**. The header names the algorithm, the payload holds **claims** (sub, iat, exp, iss, aud, plus custom ones such as roles), and the signature proves it was not changed.
- It is **signed, not encrypted** by default. Anyone can read the payload, so **never put secrets or sensitive personal data in it**.

- **HS256:** one shared secret both signs and verifies. Simple, but every verifying service holds the secret. **RS256:** a private key signs and a public key verifies, so many services can verify without being able to issue tokens. Prefer RS256 across services.

The flow

1. The client sends credentials to POST /auth/login.
2. The server authenticates through AuthenticationManager and issues a short-lived **access token** (5 to 15 minutes), and optionally a **refresh token**.
3. The client sends the access token with each call in the header **Authorization: Bearer ...**.
4. A filter validates the signature, expiry, issuer and audience, builds an Authentication with the user's authorities and puts it in the SecurityContext.
5. Authorization rules then run as normal.

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwtService;          // validates and parses the
token
    private final UserDetailsService users;
    // constructor omitted

    @Override
    protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res,
                                    FilterChain chain) throws
ServletException, IOException {
        String header = req.getHeader(HttpHeaders.AUTHORIZATION);
        if (header != null && header.startsWith("Bearer ")
                && SecurityContextHolder.getContext().getAuthentication() ==
null) {
            String token = header.substring(7);
            if (jwtService.isValid(token)) {
                UserDetails user =
users.loadUserByUsername(jwtService.username(token));
                var auth = new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);   // always continue the chain
    }
}
```

Production concerns (interviewers love these)

- **Validate everything:** signature, exp, iss and aud. **Pin the expected algorithm** and reject "none".
- **Revocation is hard** because tokens are stateless. Keep access tokens short, rotate refresh tokens, and keep a blocklist or a token version per user if you need instant logout.
- **Storage on the client:** an HttpOnly, Secure, SameSite cookie protects against script theft (XSS), but then CSRF needs attention. localStorage is easy but exposed to XSS.
- **Secrets and keys** come from a secret manager or environment variables, are long enough (at least 256 bits for HS256), and are rotated.
- **Prefer built-in support over hand-rolling:** spring-boot-starter-oauth2-resource-server validates JWTs for you with oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults())) and a JwtDecoder, and maps claims to authorities.

4. OAuth2 and OpenID Connect Basics

Roles: resource owner (the user), **client** (the app), **authorization server** (issues tokens, for example Keycloak, Okta, Auth0, Azure AD), **resource server** (your API, which validates tokens).

Grant types to know

- **Authorization Code with PKCE:** the standard for web apps, single-page apps and mobile apps. The user logs in at the authorization server, the client receives a code and exchanges it for tokens. PKCE protects the code from interception.
- **Client Credentials:** machine-to-machine calls, with no user involved. Used between backend services.
- **Refresh token:** exchanges a refresh token for a new access token.
- **Implicit and Resource Owner Password grants are deprecated** and removed in OAuth 2.1. Do not use them in new designs.

OpenID Connect (OIDC) adds an **identity layer** on top of OAuth2. OAuth2 answers "what can this app access", OIDC answers "who is the user". It adds the **ID token** (a JWT about the user), the userinfo endpoint and standard scopes such as openid, profile and email.

Other points

- **Scopes** describe what a client may do on a user's behalf. **Roles** describe what a user is. Both can end up as authorities.
- **Token types:** self-contained JWTs (validated locally, fast, hard to revoke) vs **opaque tokens** (validated by calling the authorization server's introspection endpoint, easy to revoke, slower).

- **In microservices:** the API gateway or each service acts as a resource server and validates the token, and services forward the token or use client credentials for service-to-service calls.

5. Method Security and General Web Security

Method-level security

- Enable with **@EnableMethodSecurity** (it replaces @EnableGlobalMethodSecurity).
- **@PreAuthorize** checks before the method runs, **@PostAuthorize** after it returns, using SpEL. They work through **AOP proxies**, so the self-invocation pitfall from Week 2, Day 2 applies here too.

```
@Service
public class ComplaintService {

    @PreAuthorize("hasRole('ADMIN')")
    public void delete(Long id) { /* ... */ }

    @PreAuthorize("hasRole('ADMIN') or #ownerId ==
authentication.principal.id")
    public Complaint getForOwner(Long ownerId, Long id) { /* ... */ }
}
```

Role-Based Access Control (your resume strength)

- Be ready to explain how you modeled it: users to roles, roles to permissions, whether checks are at URL level, method level or both, and how you kept data-level rules (a user sees only their own records) separate from role checks. Only describe what you actually built.

Other protections to be able to name

- **CSRF:** needed for cookie-based sessions. **CORS:** controls which browser origins may call you.
- **SQL injection:** use parameterized queries, JPA and bind parameters, never string concatenation.
- **XSS:** escape output, set Content-Security-Policy, and prefer HttpOnly cookies.
- **Security headers** (HSTS, X-Content-Type-Options, frame options), **HTTPS everywhere**, rate limiting and lockout on login, audit logging, least privilege, secrets in a vault, and dependency scanning for known vulnerabilities (OWASP Top 10).

6. REST API Best Practices

- **Resources are nouns, in plural form:** /complaints, /complaints/42/documents. Use HTTP methods for actions, not verbs in the URL.
- **Method semantics:** GET is safe and idempotent. PUT replaces a resource and is idempotent. DELETE is idempotent. PATCH partially updates. **POST creates and is not idempotent.**
- **Status codes:** 200 OK, **201 Created** (with a Location header), 204 No Content, 400 Bad Request, 401, 403, 404 Not Found, **409 Conflict** (for example a version conflict or duplicate), 422 for semantic validation errors, 429 Too Many Requests, 500, 503.
- **Idempotency keys** for POST operations that must not run twice, such as payments. The client sends a unique key, and the server stores the result and returns it on a retry.
- **Pagination, filtering and sorting** through query parameters (page, size, sort, or a cursor). Never return unbounded lists.
- **Versioning:** in the URL (/v1/...) or a header. Keep changes backward compatible, adding fields rather than changing or removing them.
- **Use DTOs, not entities,** at the API boundary, to avoid lazy-loading surprises, leaking internal fields, and tight coupling to the database.
- **Consistent error format:** use the standard **RFC 7807 Problem Details** (Spring 6 has ProblemDetail).
- **Caching and concurrency:** ETag with If-None-Match for conditional GETs, and If-Match to prevent lost updates.
- **Document** with OpenAPI (springdoc), and set timeouts and limits on request size.

7. Exception Handling

- **@RestControllerAdvice** with **@ExceptionHandler** methods gives one central place to turn exceptions into responses, keeping controllers clean.
- Extending **ResponseEntityExceptionHandler** gives you handling of standard Spring MVC exceptions, and with Problem Details enabled they produce RFC 7807 bodies.
- Define **domain exceptions** (ResourceNotFoundException, DuplicateComplaintException) and map each to a status code in one place.
- **Never leak stack traces, SQL or internal class names** to clients. Log the details server-side with a **correlation or trace id**, and return that id in the error body so support can find the log.
- **Log once** at the handler, not at every layer, and use the right level (client mistakes at WARN or INFO, unexpected errors at ERROR).

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    ProblemDetail notFound(ResourceNotFoundException ex) {
        ProblemDetail pd =
ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Resource not found");
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail invalid(MethodArgumentNotValidException ex) {
        ProblemDetail pd =
ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Validation
failed");
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        pd.setProperty("errors", errors);
        return pd;
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail unexpected(Exception ex) {
        // log with trace id here, return a generic message
        return
ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR,
"Unexpected error");
    }
}
```

8. Validation

- Add **spring-boot-starter-validation**. It uses **Jakarta Bean Validation** (Hibernate Validator).
- **Common annotations:** @NotNull, @NotBlank (strings), @NotEmpty (collections), @Size, @Min, @Max, @Positive, @Email, @Pattern, @Past, @Future.
- **@Valid on a @RequestBody** parameter triggers validation, and failures raise **MethodArgumentNotValidException**, which your handler turns into a 400 with field errors.

- **Validating path and query parameters:** put **@Validated** on the controller class and constraints on the parameters. Failures raise **ConstraintViolationException** (and in newer Spring versions HandlerMethodValidationException), so handle that too.
- **Validation groups** let the same DTO have different rules for create and update.
- **Custom constraints:** create an annotation plus a ConstraintValidator implementation for rules such as a valid case number format.
- **Validate at the boundary** (controllers), keep **business-rule checks in services**, and keep **database constraints** (not null, unique, check) as the last line of defense.

```
public record CreateComplaintRequest(
        @NotBlank @Size(max = 200) String title,
        @NotBlank @Size(max = 4000) String description,
        @NotNull Long entityId,
        @Email String contactEmail) {}

@RestController
@RequestMapping("/api/complaints")
class ComplaintController {

    @PostMapping
    ResponseEntity<ComplaintResponse> create(@Valid @RequestBody
CreateComplaintRequest req) {
        ComplaintResponse created = service.create(req);
        return ResponseEntity.created(URI.create("/api/complaints/" +
created.id())).body(created);
    }
}
```

9. Hands-On: Build a Small Secured API (1 hour)

Build this in a fresh Spring Boot 3 project. It becomes the base for the Week 3 to 4 mini project.

1. **Endpoints:** POST /auth/login, then GET, POST, PUT and DELETE on /api/complaints.
2. **Users and roles:** two in-memory or database users, one with USER and one with ADMIN, with BCrypt-hashed passwords.
3. **JWT:** issue a 15-minute access token on login, and validate it in a filter (or with the resource server support).
4. **Rules:** USER can read and create, only ADMIN can delete. Add one @PreAuthorize rule at method level.
5. **Validation:** a DTO record with at least four constraints.

6. **Errors:** a @RestControllerAdvice returning ProblemDetail for 400, 401, 403, 404 and 500.
7. **Tests:** use MockMvc with the spring-security-test dependency. @WithMockUser(roles = "ADMIN") for authorized calls, and assert that an unauthenticated call gives 401 and a USER delete gives 403.
8. **Check the filter chain** by setting logging.level.org.springframework.security=DEBUG, and read which filters run for a request.

Push the project to GitHub. It is the first piece of your portfolio.

10. Interview Questions to Answer Aloud

1. Explain the Spring Security filter chain. Where does your JWT filter go, and why?
2. What is the difference between authentication and authorization, and between 401 and 403?
3. What are SecurityContextHolder, Authentication, AuthenticationManager and UserDetailsService?
4. Role vs authority: what does the ROLE_ prefix do?
5. How would you secure a stateless REST API with JWT, step by step?
6. What is inside a JWT? Is it encrypted? What should never go into it?
7. HS256 vs RS256: when do you choose each?
8. How do you handle JWT expiry, refresh and logout?
9. Authorization Code with PKCE vs Client Credentials: when is each used?
10. What is the difference between OAuth2 and OpenID Connect?
11. When can you disable CSRF protection, and when must you not?
12. How does @PreAuthorize work, and why can it fail to apply?
13. Which HTTP methods are idempotent, and how do you make POST safe to retry?
14. How do you design consistent error responses and validate input in Spring Boot?

11. End-of-Day Checklist

- [ ] I can draw the filter chain and say where authentication and authorization happen.
- [ ] I can write a SecurityFilterChain bean in the Spring Security 6 style from memory.
- [ ] I can explain the JWT flow, its limits, and how I would handle refresh and revocation.
- [ ] I can explain Authorization Code with PKCE, Client Credentials and OIDC in simple words.
- [ ] I built the mini secured API with tests for 401 and 403.

- [ ] I can list REST status codes, idempotency rules and the Problem Details format.
- [ ] I redid four DSA problems from this week.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 2, Day 7, weekend): revise Weeks 1 and 2, take a mock interview on Java and Spring, and design a REST API for a complaint system and explain it aloud.